

IPRE-DICOM

API para análisis de imágenes médicas
Arquitectura, metodología y evaluación experimental

Avance de investigación

Pía Ayala · Septiembre de 2026

Prototipo local · Experimento con PneumoniaMNIST

Problema y pregunta de investigación

Pregunta central

¿Cómo integrar lectura multiformato, revisión técnica y modelos de clasificación en una API que otro programa pueda utilizar?

Necesidad de ingeniería

- ▶ Interpretar archivos con tamaños y representaciones diferentes.
- ▶ Separar lectura, análisis y modelos para poder probar cada componente.

Pregunta experimental

- ▶ Entrenar un clasificador con imágenes etiquetadas y un protocolo explícito.
- ▶ Comprobar su rendimiento en test y servir el checkpoint mediante HTTP.

Interpretación

El estudio presenta un prototipo de investigación. La utilidad clínica requiere una evaluación independiente.

Objetivo inicial y alcance alcanzado

Necesidad original	Estado actual y evidencia
Formatos, ruido y rotación	Lectura multiformato, métricas y checkpoints previos ejecutados.
Frontal o lateral	Generador de etiquetas IU y entrenador preparados; sin modelo entrenado.
Anonimato	Auditoría y seudonimización de metadata DICOM; sin OCR de píxeles.
Clasificador demostrable	PneumoniaMNIST: normal/neumonía, una época completa y test.
Edad, similitud y clustering	Ideas iniciales; aún no implementadas ni evaluadas.

Qué es la API y qué recibe un usuario

Concepto

Una API expone operaciones de un programa. El cliente envía una solicitud HTTP y recibe datos estructurados, normalmente JSON.

Interacción del usuario

- ▶ El visor, Swagger o un script adjuntan una imagen.
- ▶ POST /predict devuelve indicadores, privacidad y predicciones disponibles.

Papel del aprendizaje

- ▶ Un script separado ajusta los pesos del modelo con imágenes etiquetadas.
- ▶ La API carga esos pesos y predice. Una solicitud no vuelve a entrenar el modelo.

Interpretación

El navegador es un cliente del servicio. Otra aplicación puede consumir la misma API.

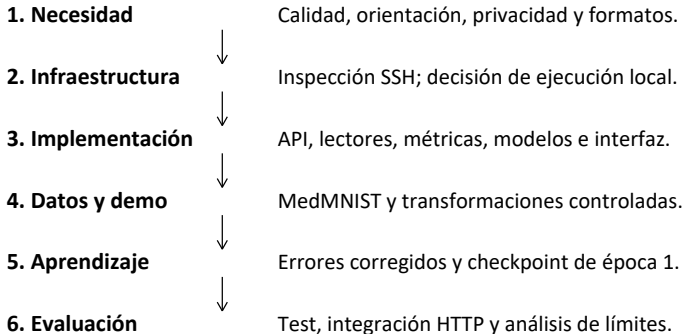
Dos preguntas, dos rutas de análisis

Ruta	Pregunta	Respuesta
POST /predict	¿Qué características técnicas y alertas tiene la imagen?	Métricas, privacidad, ruido/rotación e informe de reglas.
POST /predict/medmnist/{flag}	¿Qué clase del experimento PneumoniaMNIST predice el modelo?	Probabilidad normal y probabilidad neumonía.
GET muestra MedMNIST	¿Qué imagen y etiqueta contiene el dataset?	PNG de la muestra y etiqueta conocida.

Interpretación

Clasificar neumonía demuestra el ciclo de aprendizaje e integración; no valida ruido o rotación.

Recorrido del proyecto: hitos y dependencias



Interpretación

El despliegue en CalFuco sigue pendiente. Los resultados actuales provienen del computador local.

Etapas 1 y 2: necesidad e infraestructura

Etapa	Acción y motivo	Resultado
1. Definir alcance	Separar calidad, orientación, privacidad y clasificación: necesitan etiquetas y evaluaciones distintas.	Conjunto de servicios y tareas identificados.
2. Inspeccionar Calfuco	Revisar GPU, RAM, discos, Python y permisos mediante SSH.	RTX 3090 24 GB, dos 2080 Ti y 125 GB RAM reportados.
Decisión de ejecución	El acceso a workspace y Docker falló; Python del servidor era 3.6.9.	Continuar el prototipo local; conservar plan de despliegue.

Etapas 3 y 4: implementar y preparar datos

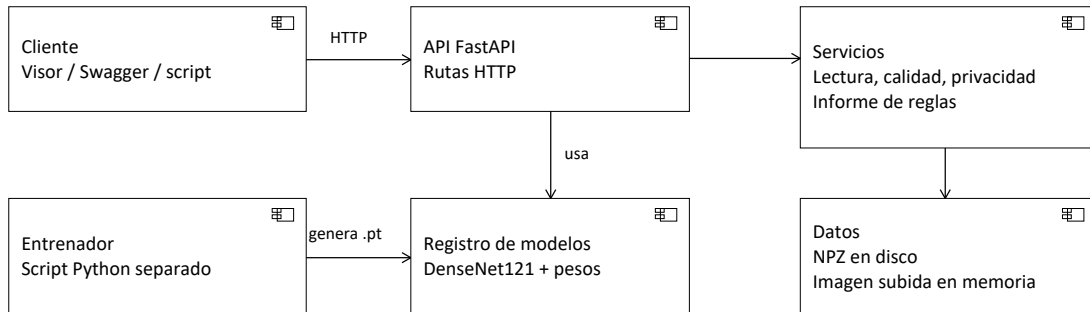
Etapa	Qué se construyó	Por qué fue necesario
3. Servicio modular	FastAPI, visor, lectores, métricas, auditoría e inferencia.	Unificar operaciones tras una interfaz HTTP.
4. Dataset conocido	Descarga de PneumoniaMNIST, catálogo y explorador de muestras.	Contar con imágenes, etiquetas y particiones reproducibles.
Demo controlada	Original del test, copia rotada, copia ruidosa y volumen NPY.	Observar respuestas y fallos en condiciones conocidas.

Etapas 5 y 6: entrenar, corregir y evaluar

Paso observado	Problema o resultado	Decisión aplicada
Primera ejecución	Memoria compartida bloqueada al usar procesos auxiliares.	DataLoader con num_workers=0.
Entrada 28 × 28	DenseNet121 reducía el mapa espacial hasta tamaño inválido.	Redimensionar a 64 × 64; conservar fuente 28 × 28.
Corrida de 5 épocas	Completó época 1; se interrumpió durante época 2.	Evaluar el checkpoint guardado de la época 1.
Integración binaria	La salida positiva aparecía rotulada como normal.	Devolver normal=1-p y neumonía=p; añadir prueba.

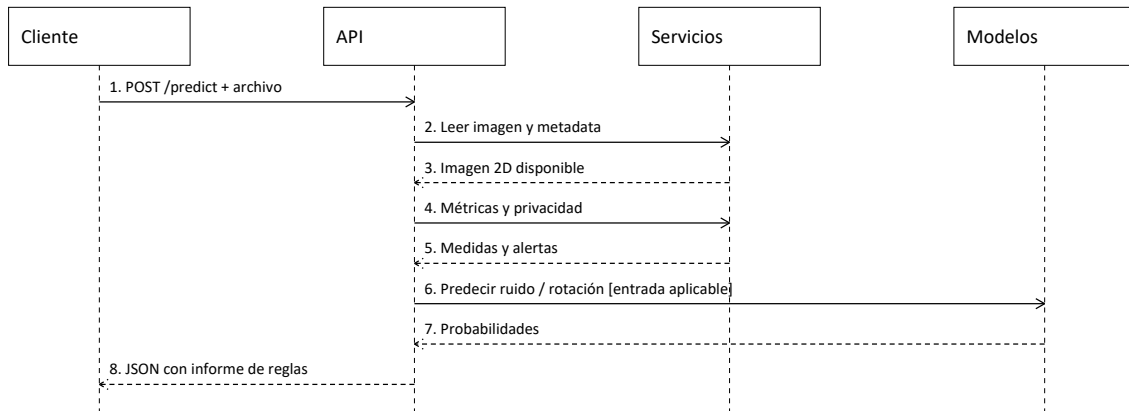
UML de componentes: dónde vive cada responsabilidad

Cada rectángulo es un componente; las flechas indican quién utiliza a quién.



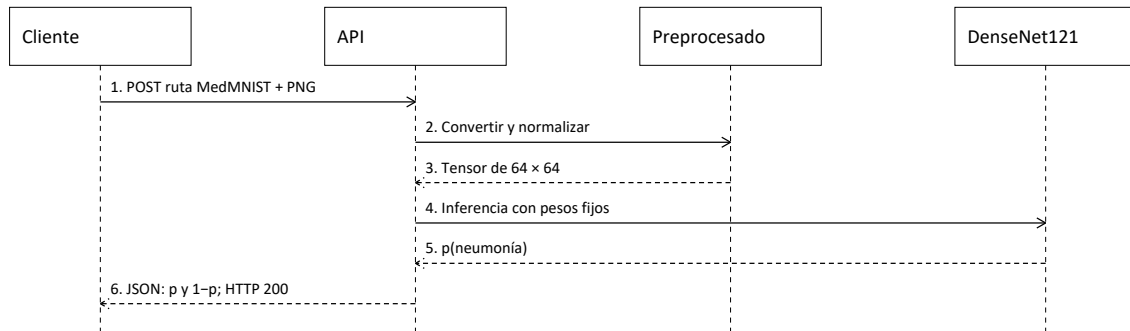
La API corre localmente. Calfuco aún no aloja este sistema. El entrenamiento produce archivos que la API carga después.

UML de secuencia: análisis técnico de una imagen



Lectura: el tiempo avanza de arriba abajo. Flecha continua = solicitud; flecha discontinua = respuesta.

UML de secuencia: usar el modelo de neumonía



El modelo ya está cargado. Esta petición no entrena ni actualiza pesos.

Lectura: el tiempo avanza de arriba abajo. Flecha continua = solicitud; flecha discontinua = respuesta.

Responsabilidad de cada módulo

Módulo	Responsabilidad	Producto que entrega
main.py + visor HTML	Recibir solicitudes y coordinar servicios.	HTTP, interfaz y JSON.
image_io.py + lector DICOM	Interpretar archivo y seleccionar representación 2D.	Imagen normalizada y metadata resumida.
quality.py / privacy.py	Calcular indicadores y buscar campos identificadores.	Medidas y alertas.
medmnist_models.py	Cargar pesos y ejecutar el clasificador del experimento.	Probabilidades por clase.
agent_workflow.py	Convertir hallazgos en recomendaciones mediante reglas.	Informe técnico.
scripts/train*.py	Ajustar y evaluar modelos fuera del servidor HTTP.	Checkpoint y métricas.

main.py: entrada y coordinación del servicio

Responsabilidad

Recibe solicitudes HTTP y decide qué funciones ejecutar. FastAPI define las rutas y Uvicorn ejecuta el servidor.

Entrada y procedimiento

- ▶ Recibe el archivo como bytes. En `/predict` comprueba que no esté vacío y no supere 64 MiB.
- ▶ Identifica el formato, llama al lector y reúne indicadores, privacidad y modelos aplicables.
- ▶ Al arrancar carga los checkpoints. `/health` informa el dispositivo y los modelos disponibles.

Recursos y resultado

- ▶ Usa FastAPI, PyTorch, MONAI y los servicios de api/. Selecciona CUDA, MPS o CPU según disponibilidad.
- ▶ Devuelve JSON o un error HTTP explicativo. Las rutas de muestras y exportación devuelven archivos.
- ▶ No ajusta pesos. `/predict` y `/predict/medmnist/{flag}` ejecutan análisis diferentes.

Interpretación

Ejemplo: una imagen NPY recibe indicadores, pero no pasa por los clasificadores generales de tórax.

Visor HTML: interacción desde el navegador

Responsabilidad

`api/static/index.html` es el cliente visual. Permite seleccionar una imagen y consultar la API sin escribir una solicitud HTTP manualmente.

Qué hace y cómo

- ▶ JavaScript recoge el archivo y lo adjunta a FormData bajo el campo file.
- ▶ El botón Analizar usa fetch para enviar POST `/predict` y presenta sus resultados.
- ▶ El explorador solicita una muestra MedMNIST y muestra la imagen y su etiqueta conocida.

Recursos y límites

- ▶ HTML estructura la pantalla, CSS define su aspecto y JavaScript gestiona las peticiones.
- ▶ La vista previa depende de los formatos que pueda mostrar el navegador. No es un visor 3D.
- ▶ El explorador no clasifica la muestra. La ruta de clasificación MedMNIST se prueba desde `/docs` o un script.

Interpretación

El navegador muestra resultados. Los modelos se ejecutan en el backend, no dentro del HTML.

image_io.py: lectura multiformato y conversión a 2D

Responsabilidad

Convierte archivos raster, NIfTI o NumPy en una representación común: una imagen 2D en escala de grises y metadata técnica.

Qué hace y cómo

- ▶ Pillow abre PNG, JPEG y otros raster. Selecciona el primer frame y convierte a grises.
- ▶ NumPy carga .npy con `allow_pickle=False`. NiBabel lee NIfTI y gzip descomprime .nii.gz.
- ▶ En arreglos de más de dos dimensiones toma cortes centrales sucesivos hasta obtener 2D.

Normalización y salida

- ▶ Para arreglos, sustituye valores no finitos y ajusta intensidades con percentiles 1 y 99.
- ▶ Entrega `LoadedImage`: imagen Pillow, familia del formato y metadata como forma y cortes elegidos.
- ▶ No interpreta anatomía. .npz se reserva para el servicio de datasets, no para una imagen individual.

Interpretación

Un volumen se reduce a un corte para este análisis. No se analiza automáticamente todo su contenido 3D.

Lector DICOM: píxeles y metadata clínica

Responsabilidad

La lectura DICOM está implementada dentro de `main.py`. `pydicom` interpreta el archivo y permite acceder a `PixelData` y sus atributos.

Procesamiento de píxeles

- ▶ Detecta la marca DICM o la extensión `.dcm/.dicom` y abre los bytes con `pydicom`.
- ▶ `read_dicom_array` obtiene `pixel_array`, toma el primer frame y aplica `RescaleSlope` e `Intercept`.
- ▶ Aplica la ventana disponible o el rango mínimo/máximo. Invierte `MONOCHROME1` y convierte a 8 bits.

Recursos y salida

- ▶ `pydicom` decodifica el objeto. `NumPy` transforma intensidades y `Pillow` crea la imagen.
- ▶ Entrega imagen y metadata resumida: modalidad, filas, columnas y posición de vista.
- ▶ La API deriva el dataset DICOM a `privacy.py`. Si `PixelData` no se decodifica, devuelve un error HTTP.

Interpretación

La lectura DICOM no es un modelo de IA. Prepara los datos que después consumen los otros módulos.

quality.py: indicadores calculados sin aprendizaje

Responsabilidad

Describe numéricamente la imagen 2D que entrega el lector. No carga archivos .pt ni necesita ejemplos etiquetados.

Cómo calcula los indicadores

- ▶ Convierte a grises y divide las intensidades por 255 para trabajar entre 0 y 1.
- ▶ Brillo: media de intensidad. Contraste: desviación estándar de los píxeles.
- ▶ Ruido: mediana de la diferencia absoluta frente a una copia suavizada con BoxBlur(1).

Recursos y producto

- ▶ Nitidez: promedio de las varianzas de diferencias entre píxeles en ambos ejes.
- ▶ Usa Pillow para grises y suavizado, y NumPy para las operaciones numéricas.
- ▶ Devuelve dimensiones y medidas redondeadas. No produce por sí solo una etiqueta buena/mala.

Interpretación

noise_estimate es una fórmula descriptiva. P(ruido) es la salida de otro componente: un clasificador con pesos.

Responsabilidad

Revisa una lista definida de campos DICOM y una declaración de texto incrustado. No elimina datos ni examina visualmente los píxeles.

Reglas aplicadas

- ▶ Busca campos no vacíos: PatientName, PatientID, fecha de nacimiento e institución.
- ▶ Si BurnedInAnnotation no declara NO, solicita revisión de texto en píxeles.
- ▶ En raster no evalúa metadata DICOM. La revisión de píxeles queda pendiente.

Recursos y resultado

- ▶ Aplica reglas Python al dataset abierto por pydicom, sin red neuronal.
- ▶ Devuelve nombres y conteo de campos y avisos. Omite los valores sensibles.
- ▶ El campo anonymous depende de estas reglas limitadas. No garantiza anonimato.

Interpretación

Ausencia de campos conocidos y BurnedInAnnotation=NO no equivalen a una inspección real de los píxeles.

anonymize.py: copia DICOM seudonimizada

Responsabilidad

Modifica una copia del dataset para retirar identificadores conocidos. La ruta `/anonymize/dicom` permite descargar esa copia.

Condiciones y pasos

- ▶ Exige `DICOM_PSEUDONYM_SECRET` de al menos 16 caracteres y `BurnedInAnnotation` igual a `NO`.
- ▶ Hace una copia profunda, elimina tags privados y vacía los campos identificadores listados.
- ▶ Sustituye UUIDs seleccionados mediante HMAC-SHA256 y actualiza el UUID del archivo.

Recursos y producto

- ▶ Usa `pydicom`, `copy`, `hashlib` y `hmac`. La misma clave permite un remapeo consistente de un UUID.
- ▶ Devuelve la copia y un resumen de las operaciones. `main.py` la serializa como archivo DICOM.
- ▶ Los píxeles no cambian. No hay OCR, redacción visual ni validación institucional del perfil.

Interpretación

Es un perfil experimental de seudonimización, no una certificación de desidentificación completa.

Responsabilidad

Administra el catálogo, la descarga explícita y la extracción de muestras. El dataset aporta imágenes y etiquetas, no pesos de un modelo.

Entrada y procedimiento

- ▶ Recibe identificador del dataset, partición train/val/test, resolución e índice de muestra.
- ▶ Valida esos parámetros y usa la clase correspondiente de la biblioteca medmnist.
- ▶ `sample_png` toma una imagen y su etiqueta, y convierte la imagen a PNG mediante Pillow.

Archivos y salida

- ▶ Lee `MEDMNIST_ROOT` o `data/medmnist`. Descarga solo ante una petición explícita.
- ▶ Devuelve PNG, etiqueta y conteo. La API envía la etiqueta y el conteo en cabeceras HTTP.
- ▶ Admite ChestMNIST y PneumoniaMNIST. Solo usamos el segundo en el experimento.

Interpretación

Archivo utilizado: `data/medmnist/pneumoniamnist.npz`. Ver una muestra no ejecuta una clasificación.

medmnist_models.py: inferencia con el modelo guardado

Responsabilidad

Reconstruye la red entrenada y aplica sus pesos a una imagen nueva. Es el componente que predice normal o neumonía en el experimento actual.

Carga y cálculo

- ▶ Busca `models/medmnist_*.pt`. Lee dataset, tamaño, número de salidas y `state_dict`, que contiene los pesos.
- ▶ Crea `DenseNet121` de MONAI, carga los pesos y activa `model.eval()`.
- ▶ Convierte a grises, redimensiona y normaliza. Ejecuta PyTorch con `no_grad()`, sin ajustar parámetros.

Recursos y producto

- ▶ Pillow y NumPy preparan la imagen. MONAI define la red y PyTorch realiza el cálculo.
- ▶ Sigmoid transforma la salida en $p(\text{neumonía})$. Para dos clases devuelve también $p(\text{normal})=1-p$.
- ▶ Entrega etiquetas, probabilidades, tamaño de entrada y advertencia de uso experimental.

Interpretación

El checkpoint actual usa entrada 64×64 . Falta unificar el filtro de redimensionado con el entrenamiento.

Responsabilidad

Recibe los resultados ya calculados y redacta recomendaciones predefinidas. No interpreta la imagen mediante un modelo de lenguaje.

Reglas de decisión

- ▶ Si la probabilidad de ruido o rotación alcanza 0,5, añade un hallazgo y recomienda revisión manual.
- ▶ Si hay identificadores en metadata, recomienda desidentificarlos antes de compartir.
- ▶ Si los píxeles necesitan revisión, recomienda OCR. El módulo no ejecuta ese OCR.

Orquestación y salida

- ▶ Con LangGraph instalado ejecuta START, un nodo `technical_report` y END.
- ▶ Sin LangGraph llama directamente a la misma función `build_report`. El criterio no cambia.
- ▶ Devuelve status, findings y `recommended_actions`. No usa un LLM, búsqueda web ni entrenamiento.

Interpretación

Aquí “agente” designa un flujo determinista de un nodo. No existe un agente autónomo de diagnóstico.

train_medmnist.py: aprendizaje y evaluación

Responsabilidad

Es un programa separado de la API. Recibe el dataset etiquetado y una configuración, y ajusta una DenseNet121 para la tarea elegida.

Cómo aprende

- ▶ Carga splits oficiales con MedMNIST. torchvision redimensiona, convierte a tensor y normaliza.
- ▶ DataLoader organiza lotes. BCEWithLogitsLoss mide error y Adam actualiza pesos mediante gradientes.
- ▶ Evalúa validación sin gradientes y guarda un checkpoint cuando mejora el ROC-AUC.

Recursos y producto

- ▶ PyTorch y MONAI realizan aprendizaje. scikit-learn calcula AUC, F1 y accuracy por etiqueta.
- ▶ Al finalizar, evalúa el mejor checkpoint en test y escribe métricas e historial. W&B es opcional.
- ▶ Produce .pt con pesos y metadata, y JSON de resultados. La API carga el .pt al arrancar.

Interpretación

La corrida real se interrumpió en época 2. Se evaluó después el checkpoint de época 1 y se conservaron sus resultados.

train.py: entrenador general de ruido, giro o vista

Responsabilidad

Prepara entrenamiento binario para una tarea seleccionada: noise, rotation o view. Su entrada es un CSV de rutas y etiquetas.

Cómo está implementado

- ▶ Lee path, la etiqueta de la tarea y, si existe, source_id para agrupar originales y variantes.
- ▶ Divide train/validación por grupos. Sin grupos usa división estratificada y advierte riesgo de fuga.
- ▶ MONAI carga imágenes y prepara tensores. DenseNet121 usa dos salidas, CrossEntropyLoss y Adam.

Recursos y alcance

- ▶ scikit-learn mide accuracy, balanced accuracy, F1 y AUC. Guarda best_{task}.pt si mejora AUC.
- ▶ Puede registrar parámetros y métricas en W&B cuando se habilita ese modo.
- ▶ Tener el entrenador no demuestra que los checkpoints previos provengan de una corrida documentada aquí.

Interpretación

Los modelos previos de ruido/rotación se probaron en inferencia. Frontal/lateral todavía no tiene entrenamiento demostrado.

export_wandb_results.py: publicación de resultados existentes

Responsabilidad

Importa a W&B las métricas guardadas del experimento. No abre imágenes, no entrena una red y no recalcula el rendimiento.

Entrada y procedimiento

- ▶ Lee `models/medmnist_pneumoniarnist_28_metrics.json`.
- ▶ `wandb.init` crea un registro offline marcado como importación posterior. `run.log` guarda el punto de época 1.
- ▶ `run.summary` conserva las métricas de test y `log_artifact` adjunta el JSON como artefacto de evaluación.

Recursos y resultado

- ▶ Usa `json`, `pathlib` y el SDK de W&B. Escribe el registro local en `output/wandb`.
- ▶ `wandb sync` transmite ese registro a la cuenta autenticada. La sincronización del run `oliy37yu` está confirmada.
- ▶ Esta importación publica métricas y configuración. No incluye imágenes ni el checkpoint del modelo.

Interpretación

El panel documenta una época ya evaluada. No representa cinco épocas completas ni seguimiento en vivo de la corrida original.

Herramientas del servicio y su función

Herramienta	Qué es	Uso concreto
Python	Lenguaje del backend.	Implementa lectura, modelos, endpoints y pruebas.
FastAPI / Uvicorn	Framework web / servidor HTTP.	Define rutas y recibe solicitudes locales.
HTML, CSS, JavaScript	Tecnologías del navegador.	Selección de archivo, envío y visualización de resultados.
Pillow / NumPy	Imágenes y arreglos numéricos.	Escala de grises, tamaños y operaciones sobre píxeles.
pydicom	Biblioteca para DICOM.	Lee metadata y píxeles; exporta copia seudonimizada.

Herramientas de aprendizaje y experimentación

Herramienta	Papel real	Estado
PyTorch	Tensores, gradientes y optimización de pesos.	Entrenamiento e inferencia ejecutados.
MONAI	Biblioteca que aporta DenseNet121 y transformaciones.	Arquitectura usada; MONAI no es un modelo por sí mismo.
MedMNIST	Datasets con muestras, etiquetas y splits.	PneumoniaMNIST instalado y utilizado.
scikit-learn	Funciones para calcular métricas y separar grupos.	Evaluación y pruebas.
W&B	Registro de experimentos, parámetros y artefactos.	Integración y registro offline; métricas reales importadas y sincronizadas.
LangGraph	Orquestación del informe mediante un grafo.	Un nodo determinista; sin LLM ni aprendizaje.

Funcionamiento paso a paso de POST /predict

Procedimiento

1. El cliente adjunta el archivo en el campo file. La API rechaza entradas vacías y archivos mayores de 64 MiB.
- ▶ 2. El lector interpreta el formato y obtiene una imagen 2D. Conserva información como dimensiones, modalidad o forma del volumen.
- ▶ 3. Calcula indicadores en la imagen resultante y revisa privacidad. Prepara además una copia de 224×224 para los modelos previos.
- ▶ 4. Para raster o DICOM CR/DX ejecuta los clasificadores disponibles. Otras entradas reciben métricas sin esos clasificadores.
- ▶ 5. El informe reúne hallazgos y recomendaciones. La respuesta JSON regresa al navegador o al programa solicitante.

Lectura y visualización: capacidades y límites

Entrada	Tratamiento actual	Límite relevante
DICOM (.dcm/.dicom)	PixelData, slope/intercept, ventana e inversión MONOCHROME1.	Primer frame; no visor volumétrico.
PNG/JPEG/TIFF/BMP/WebP	Lectura raster y escala de grises.	Vista previa nativa del navegador según formato.
NIfTI (.nii/.nii.gz)	Lectura y selección de cortes centrales hasta 2D.	La interfaz no ofrece navegación 3D.
NumPy (.npy)	Arreglos sin objetos serializados; selección 2D.	Fuera del dominio de clasificadores de tórax.
MedMNIST (.npz)	Archivo de dataset consultado mediante rutas dedicadas.	No se sube como radiografía individual.

Qué significan las medidas de calidad

Indicador	Cálculo implementado	Interpretación permitida
Brillo	Promedio de intensidad entre 0 y 1.	Compara claridad global.
Contraste	Desviación estándar de intensidad.	Describe dispersión de tonos.
Ruido estimado	Mediana del cambio frente a un suavizado local.	Compara variaciones de alta frecuencia.
Nitidez estimada	Promedio de varianzas de diferencias en dos ejes.	Describe variaciones de bordes.

Interpretación

Se calculan con fórmulas y no necesitan entrenamiento. No hay umbral clínico validado.

Privacidad e informe de recomendaciones

Alcance

El sistema revisa metadata DICOM y genera un informe por reglas. La revisión de privacidad no garantiza anonimato completo.

Metadata DICOM

- ▶ La auditoría informa nombres y conteos de campos identificadores, sin devolver sus valores.
- ▶ La copia seudonimizada vacía campos, retira tags privados y remapea UUIDs con una clave.

Píxeles e informe

- ▶ Los píxeles permanecen intactos. No hay OCR ni borrado de texto incrustado.
- ▶ LangGraph organiza un nodo determinista de recomendaciones. No hay LLM ni agente autónomo.

Interpretación

Una alerta exige revisión humana. No se puede declarar que una imagen carece de información identificadora.

Dataset del experimento: PneumoniaMNIST

Unidad de observación

Cada muestra contiene una radiografía en escala de grises y una etiqueta conocida: 0 = normal, 1 = neumonía.

Datos utilizados

- ▶ Archivo oficial local de 28×28 píxeles.
- ▶ 5.856 imágenes, distribuidas en entrenamiento, validación y test.
- ▶ La etiqueta permite comparar una predicción con la respuesta conocida.

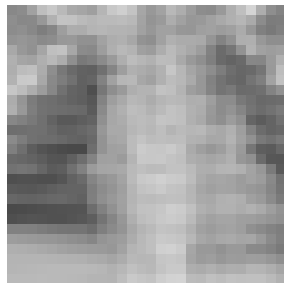
Adecuación al proyecto

- ▶ Permite demostrar entrenamiento, evaluación e inferencia HTTP.
- ▶ No contiene etiquetas de ruido, rotación, privacidad o edad.
- ▶ Redimensionar a 64×64 no agrega detalle anatómico.

Interpretación

Este experimento valida un recorrido técnico con un dataset conocido, no todas las tareas iniciales.

Una muestra: imagen más respuesta conocida



Etiqueta 0: Normal

Test, índice 1 · fuente 28×28



Etiqueta 1: Neumonía

Test, índice 0 · fuente 28×28

Imagen

Los píxeles son la entrada del modelo.

Etiqueta

La respuesta conocida sirve para aprender o evaluar.

Las imágenes se amplían solo para mostrarlas. La etiqueta proviene del dataset, no de una interpretación visual realizada aquí.

Por qué dividimos las imágenes en tres grupos

5.856 imágenes etiquetadas: se asigna una función distinta a cada grupo.



APRENDER

4.708 imágenes · 80,4%

Train: las imágenes y sus etiquetas permiten corregir los pesos.

REVISAR

524 imágenes · 8,9%

Validación: comprobar el modelo al final de la época, sin modificarlo.

EXAMINAR

624 imágenes · 10,7%

Test: medir el checkpoint elegido con imágenes separadas.

“Ajustar pesos” significa cambiar los números internos del modelo para reducir errores. En validación y test esos números permanecen fijos.

Distribución de clases: qué significa desbalance

Gris = normal; blanco = neumonía. Cada barra representa el 100% de su grupo.

Train



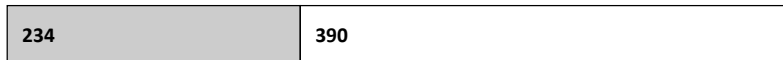
25.8% normal; 74.2% neumonía

Validación



25.8% normal; 74.2% neumonía

Test



37.5% normal; 62.5% neumonía

En test hay más neumonías: responder siempre “neumonía” ya acierta $390/624 = 62,5\%$. Por eso necesitamos una comparación y métricas por clase.

Separación de datos y prevención de fuga

Fuga de información

Ocurre cuando información de evaluación influye en el aprendizaje o la selección del modelo y hace que el resultado parezca mejor de lo que generaliza.

Protocolo utilizado

- ▶ Train actualiza pesos. Validación decide el checkpoint. Test evalúa el modelo elegido.
- ▶ Se conservaron los splits oficiales de PneumoniaMNIST. No se realizó una auditoría propia por paciente del NPZ.

Control y riesgo pendiente

- ▶ El entrenador general admite `source_id` para agrupar una imagen y sus variantes.
- ▶ Test #0 también forma parte de la demo. Si sus resultados guían ajustes, se necesitará un nuevo test externo reservado.

Interpretación

Las copias de una misma imagen deben permanecer en la misma partición.

Qué contiene el modelo y qué se guarda

Modelo

DenseNet121 transforma píxeles en características y produce una salida numérica. Los pesos son los parámetros internos que aprende a ajustar.

Representación y aprendizaje

- ▶ Las capas aprenden filtros a partir de ejemplos etiquetados.
- ▶ Las conexiones densas reutilizan características anteriores.
- ▶ La última salida, llamada logit, no está restringida al intervalo de 0 a 1.

Salida y checkpoint

- ▶ Sigmoid convierte el logit en $p(\text{neumonía})$.
- ▶ Con umbral 0,5 se obtiene la clase predicha.
- ▶ El checkpoint guarda pesos y metadata para reconstruir la red sin entrenarla otra vez.

Interpretación

MONAI aporta la implementación de la arquitectura. DenseNet121 es el modelo concreto utilizado.

Cómo aprende DenseNet121

1. Datos

Leer un lote de 64 imágenes de train con sus etiquetas.



2. Predicción

La red calcula un valor de salida a partir de los píxeles.



3. Error

BCEWithLogitsLoss compara salida y etiqueta conocida.



4. Aprendizaje

Backpropagation calcula gradientes; Adam ajusta pesos.



5. Validación

Sin actualizar pesos, calcular métricas en validation.



6. Checkpoint

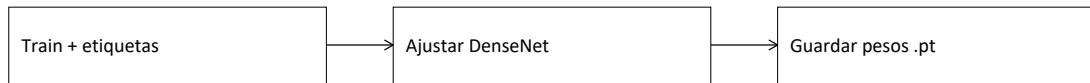
Guardar pesos cuando mejora el ROC-AUC de validación.

Interpretación

Una época recorre todo train. Durante inferencia se usan pesos fijos y no se propaga el error.

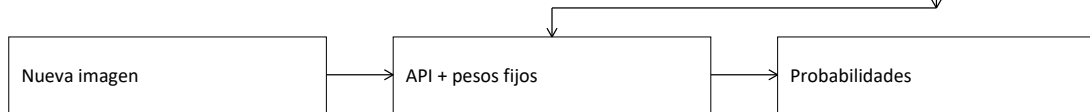
Entrenamiento e inferencia: dos momentos diferentes

ENTRENAMIENTO: aprende a partir de imágenes etiquetadas



La API carga este archivo al arrancar

INFERENCIA: aplica los pesos ya aprendidos



Enviar una imagen a la API no enseña al modelo. Para aprender de nuevas imágenes hay que ejecutar otro entrenamiento.

Configuración exacta de la corrida inicial

Parámetro	Valor observado o configurado
Modelo	MONAI DenseNet121, 2D, un canal de entrada y una salida; sin pesos preentrenados solicitados.
Entrada	28×28 redimensionada a 64×64 ; intensidades normalizadas a aproximadamente $[-1, 1]$.
Optimización	BCEWithLogitsLoss, Adam, learning rate inicial 0,0001, batch de 64.
Plan	5 épocas; CosineAnnealingLR configurado para ese horizonte; semilla 42.
Ejecución real	CPU local, num_workers=0; una época completa y detención durante la segunda.
Modelo evaluado	Checkpoint guardado al terminar la época 1. No se evaluaron los pesos parciales de época 2.

Cómo interpretar las métricas del experimento

Métrica	Significado para este estudio
Accuracy	Fracción de imágenes cuya clase coincide con la etiqueta, usando umbral 0,5.
F1 macro	Media del F1 de normal y neumonía. F1 combina precisión y sensibilidad por clase.
ROC-AUC	Mide cómo ordena positivos frente a negativos al variar el umbral. 0,5 es referencia aleatoria.
Loss	Error usado para ajustar pesos. No es un porcentaje de imágenes incorrectas.
Probabilidad individual	Salida del modelo para una muestra; no equivale a accuracy global ni certeza clínica.

Interpretación

La correlación no se calculó: el experimento evalúa clasificación binaria.

Resultados medidos del checkpoint de época 1

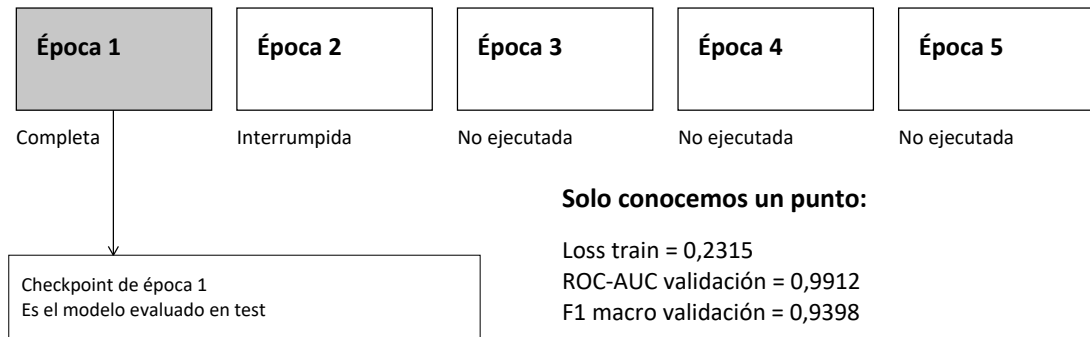
Medición	Validación (524)	Test (624)	Lectura
ROC-AUC	0,9912	0,9478	Descenso de 0,0434 en test.
F1 macro	0,9398	0,8337	Descenso de 0,1061 en test.
Accuracy	No conservada en el registro	85,42%	533 aciertos y 91 errores.
Loss de train	No corresponde	No corresponde	0,2315 en train al terminar época 1.

Interpretación

Un punto por época no permite dibujar una curva de aprendizaje ni demostrar convergencia.

Qué se completó realmente durante la corrida

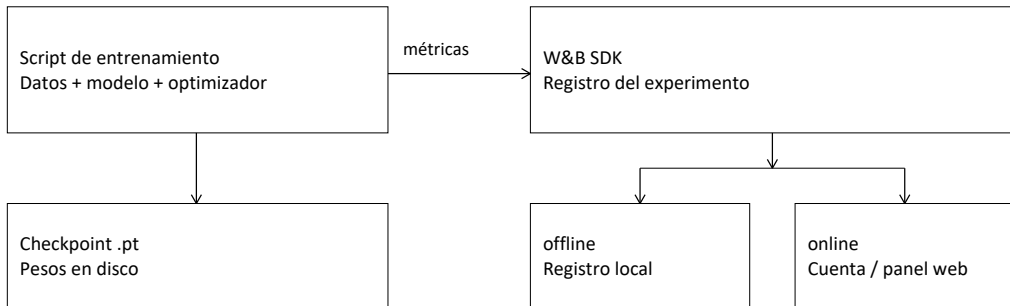
Plan: 5 épocas. Evidencia disponible: una época completa.



Un solo punto no permite evaluar convergencia ni sobreajuste. No se conserva qué fracción de la segunda época alcanzó a ejecutarse.

W&B: dónde se integra en el experimento

W&B actúa como una bitácora: recibe registros del entrenamiento.



Entrenar lo hace PyTorch. W&B registra parámetros, métricas y el archivo del modelo para consultar y comparar experimentos.

W&B: qué programamos en los entrenadores

Momento	Código integrado	Qué queda registrado
Inicio	<code>wandb.init(config=...)</code>	Modelo, semilla, épocas, lote, learning rate y dispositivo.
Fin de cada época	<code>run.log(...)</code>	Loss de train y métricas de validación.
Test de MedMNIST	<code>run.summary[...]</code>	ROC-AUC, F1 macro y accuracy finales.
Cierre	<code>wandb.Artifact + log_artifact</code>	Mejor checkpoint con alias best/latest, seguido de <code>run.finish()</code> .

Interpretación

El artefacto se registra al final: una interrupción puede impedir ese paso.

W&B: evidencia disponible y alcance del registro

Etapa	Qué ocurrió	Qué demuestra
Entrenamiento original	W&B desactivado. Una época completa y detención en la segunda.	El modelo y las métricas existen localmente.
Importación posterior	<code>export_wandb_results.py</code> leyó el JSON de métricas y creó un registro offline.	Resultados reales, sin repetir el entrenamiento.
Sincronización web	Run oliy37yu sincronizado el 28 de septiembre de 2026.	Experimento disponible en la cuenta autenticada.
Contenido publicado	Configuración, un punto de época 1, resumen de test y artefacto JSON.	Sin imágenes médicas ni checkpoint en esta importación.

Interpretación

La importación conserva resultados existentes. No reconstruye épocas ni curvas que no se registraron.

W&B: cómo consultar el experimento publicado

Registro confirmado

Proyecto ipre-dicom-medmnist. Run oliy37yu: importación posterior de las métricas guardadas del checkpoint de época 1.

Qué encontrará la audiencia

- ▶ Configuración: dataset, arquitectura, tamaños y número de muestras.
- ▶ Historial: loss 0,2315, AUC de validación 0,9912 y F1 macro 0,9398 en época 1.

Qué significa cada evidencia

- ▶ Resumen de test: accuracy 85,42%, F1 macro 0,8337, ROC-AUC 0,9478.
- ▶ Artefacto de evaluación: JSON de métricas. Un punto no describe una curva de aprendizaje.

Interpretación

Abrir experimento en W&B (enlace clicable)

Cómo leer los 624 resultados del test

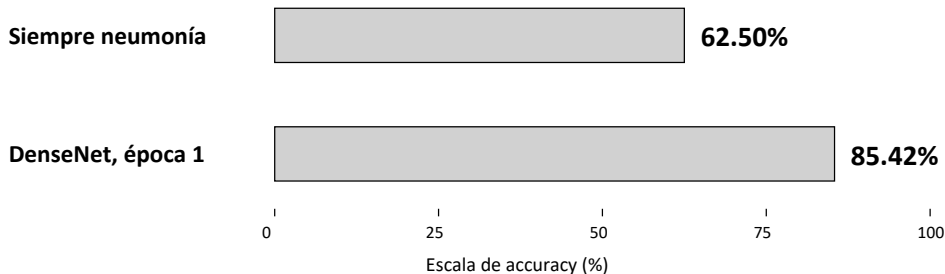
La fila indica la etiqueta conocida; la columna indica la respuesta del modelo.

	Predijo normal	Predijo neumonía
Real: normal	157 Normales reconocidos	77 Falsas alarmas
Real: neumonía	14 Neumonías omitidas	376 Neumonías detectadas

533 aciertos (157 + 376) y 91 errores (77 + 14). Detecta 96,4% de neumonías, pero reconoce solo 67,1% de normales.

Comparación: ¿el modelo supera una regla trivial?

Accuracy = porcentaje de respuestas correctas sobre las mismas 624 imágenes.



La red mejora la referencia en 22,92 puntos porcentuales. Aún no se comparó contra ResNet ni se repitió con otras semillas.

Comparación de rendimiento en el mismo test

Método	Accuracy	F1 macro	ROC-AUC
Siempre predecir neumonía	62,50%	0,3846	0,5000
DenseNet121, época 1	85,42%	0,8337	0,9478
Mejora absoluta	22,92 puntos porcentuales	0,4491	0,4478

Interpretación

La red supera una regla basada en frecuencia. Esta tabla no compara DenseNet con ResNet u otras redes.

Comparación de componentes y modelos del proyecto

Componente	Datos / salida	Qué evidencia existe
Métricas de calidad	Píxeles / medidas numéricas.	Demo reproducible; sin ajuste de pesos.
DenseNet ruido y rotación	Checkpoints previos / dos clases por tarea.	Cargan y responden; procedencia de entrenamiento no documentada.
DenseNet PneumoniaMNIST	Splits oficiales / normal-neumonía.	Una época, checkpoint y test medido.
Clasificador de vista	Manifiesto IU / frontal-lateral.	Preparación implementada; sin entrenamiento.
ResNet, ViT, CLIP, DINO, kNN	Alternativas propuestas al inicio.	Sin comparación experimental en este repositorio.

Experimento técnico de ruido y rotación

Caso	Ruido por fórmula	P(ruido)	P(rotación)
Original de MedMNIST	0,0000	0,9465	0,4261
Misma muestra rotada 25°	0,0000	0,9492	0,4278
Misma muestra con ruido añadido	0,0588	0,9943	0,4699
Volumen NPY de demostración	0,0000	No ejecutado	No ejecutado

Interpretación

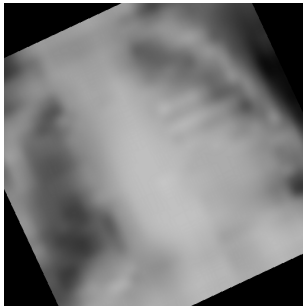
Estas variantes de una sola fuente son una prueba funcional, no un conjunto independiente de evaluación.

Prueba controlada: misma imagen, tres condiciones



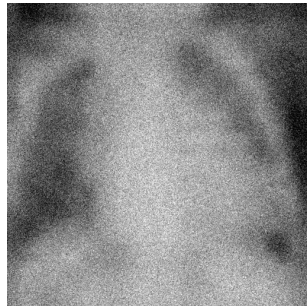
Original

Referencia de la demo



Rotada 25°

Giro artificial conocido



Ruido añadido

Perturbación gaussiana

Se comparan respuestas antes y después de modificar una misma fuente. Este ensayo descubre fallos puntuales; no mide rendimiento poblacional.

Análisis crítico de los hallazgos

Criterio de interpretación

Un resultado funcional comprueba que el programa responde. Un resultado predictivo necesita etiquetas y un conjunto de evaluación apropiado.

Calidad técnica

- ▶ El ruido calculado aumentó a 0,0588 con la perturbación añadida.
- ▶ La alerta de ruido en la original no puede llamarse falso positivo confirmado: no hay etiqueta de calidad adjudicada.

Clasificación y límites

- ▶ El modelo de rotación falló en la copia girada 25°. Un caso no permite estimar sensibilidad global.
- ▶ DenseNet de neumonía superó la referencia. El descenso entre validación y test exige cautela.

Interpretación

Las evidencias de calidad y neumonía corresponden a tareas distintas y no deben mezclarse.

Del checkpoint a una predicción HTTP

Carga del modelo

Al iniciar el servidor, el registro lee `models/medmnist_*.pt` y reconstruye DenseNet121 con los pesos y la configuración guardados.

Solicitud y cálculo

- ▶ La ruta MedMNIST recibe PNG/JPEG, convierte a grises, redimensiona y normaliza.
- ▶ El modelo calcula $p(\text{neumonía})$. La respuesta incluye $p(\text{normal}) = 1 - p(\text{neumonía})$.

Evidencia de integración

- ▶ Test #0 tiene etiqueta 1, neumonía.
- ▶ La petición devolvió HTTP 200 y $p(\text{neumonía}) = 0,98005$.
- ▶ Una predicción individual no equivale a la accuracy de todo el test.

Interpretación

La prueba confirma carga e inferencia HTTP. La equivalencia exacta del preprocesamiento sigue pendiente.

Qué puede mostrar hoy la demostración

Pantalla o acción	Qué muestra	Qué permite comprobar
GET /health	CPU y modelos cargados.	Servidor activo y checkpoint disponible.
Visor /, botón Analizar	Métricas, alertas y modelos previos.	Ruta técnica POST /predict.
Explorador MedMNIST	Muestra test #0, imagen y etiqueta.	Dataset disponible. No ejecuta clasificación.
/docs: ruta de predicción MedMNIST	Subir PNG de demo y ejecutar.	Nueva predicción del checkpoint entrenado.
python scripts/run_demo.py	Tabla y reporte JSON.	Repetición automatizada de la demo.

Guion reproducible de demostración

1. Desde la carpeta ipre-dicom, activar el entorno: `source .venv/bin/activate`
2. Iniciar el servicio: `MEDMNIST_ROOT=data/medmnist python -m uvicorn api.main:app --host 127.0.0.1 --port 8000`
3. Abrir `http://127.0.0.1:8000` y analizar los archivos de `demo_assets`: original, rotada, ruidosa y volumen NPY.
4. Abrir `http://127.0.0.1:8000/docs`. En la ruta MedMNIST usar `flag=pneumoniamnist` y subir `01_original_medmnist.png`.
5. Contrastar una probabilidad individual con la tabla agregada de las 624 imágenes de test. Explicar que miden aspectos distintos.

Verificación y amenazas a la validez

Dos niveles de evidencia

Las 27 pruebas de software verifican contratos y comportamiento del código. No constituyen una validación clínica del clasificador.

Limitaciones experimentales

- ▶ Una época completa y una semilla. Sin convergencia demostrada.
- ▶ Sin intervalos de confianza, test externo ni comparación entre arquitecturas.
- ▶ Procedencia de los modelos previos de calidad sin documentar.

Limitaciones del sistema

- ▶ Entrenamiento y API usan filtros de redimensionado no idénticos.
- ▶ La ruta técnica acepta raster sin verificar contenido torácico.
- ▶ Vista previa limitada, sin navegación 3D ni OCR.

Interpretación

Los resultados sustentan un avance de ingeniería y aprendizaje inicial, con tareas de validación abiertas.

Roadmap siguiente y criterio de término

Hito pendiente	Trabajo concreto	Evidencia esperada
1. Consistencia del pipeline	Unificar interpolación y registrar configuración/épocas de forma incremental.	Prueba de igualdad entre preprocesamientos e historial completo.
2. Experimento comparativo	Entrenar DenseNet y ResNet con protocolo equivalente y varias semillas.	Curvas, métricas por clase y tabla con variabilidad.
3. Calidad y vista	Obtener etiquetas revisadas e incorporar IU frontal/lateral.	Evaluación independiente por tarea.
4. Calfuco	Resolver carpeta, entorno y acceso autorizado a GPU.	Servicio ejecutado allí y prueba HTTP vía túnel SSH.

Conclusiones del avance

1. Se construyó una API modular que integra lectura de archivos, indicadores técnicos, revisión de metadata y modelos. Un cliente externo puede reutilizar esas operaciones por HTTP.
2. El ciclo experimental quedó demostrado con PneumoniaMNIST: entrenamiento local, checkpoint de época 1, evaluación sobre 624 muestras y predicción desde la API.
3. El checkpoint alcanzó accuracy 85,42%, F1 macro 0,8337 y ROC-AUC 0,9478. Superó la referencia constante, aunque la evidencia sigue siendo inicial.
4. Las pruebas también revelaron límites concretos: clasificadores previos poco documentados, alertas de calidad por revisar, preprocesamiento no idéntico y despliegue remoto pendiente.

Trazabilidad: dónde revisar cada afirmación

Evidencia	Archivo o ubicación
Arquitectura y contratos	api/main.py; api/static/index.html; README.md
Algoritmos del análisis	api/image_io.py; quality.py; privacy.py; anonymize.py; agent_workflow.py
Entrenamiento	scripts/train_medmnist.py; trazas de la sesión
Checkpoint y resultados	models/medmnist_pneumoniamnist_28.pt y archivo *_metrics.json
Demostración	scripts/prepare_demo.py; scripts/run_demo.py; demo_assets/demo_results.json
Pruebas y plan remoto	tests/; DEMO_PRESENTACION.md; CHEATSHEET_CALFUCO.md

Interpretación

Los conteos se obtuvieron del NPZ y los resultados del checkpoint local; no son cifras tomadas de otros estudios.